

Angular HandsOn 3: Directives and Pipes

HANDS-ON 3 [Beginner]

Directives & Pipes ? Built-in and Custom

Topics Covered:

Structural Directives ? *ngIf, *ngFor, [Check]

*ngSwitch

Attribute Directives ? ngClass, ngStyle[Check]

Custom Attribute Directive[Check] Built-in Pipes ? date, uppercase, currency,[Check]

number

Custom Pipe with PipeTransform[Check]

Directives extend HTML with Angular-specific behaviour and pipes transform displayed values. In this hands-on, you will use all built-in structural and attribute directives, and build one custom directive and one custom pipe for the Student Course Portal.

Task 1: Structural Directives

Goal: Control DOM structure dynamically using *ngIf, *ngFor, and *ngSwitch.

Steps:

25. In CourseListComponent, add a boolean property isLoading = true. Use *ngIf to show a <p>Loading courses...</p> when isLoading is true, and show the course grid when it is false. After 1.5 seconds (use setTimeout in ngOnInit), set isLoading = false.

26. Use *ngFor to render the CourseCardComponents: <app-course-card *ngFor='let course of courses; let i = index; trackBy: trackByCourseId'>. Implement the trackByCourseId(index, course) method returning course.id ? explain in a comment why trackBy improves performance.

27. Add a gradeStatus property to each course object: 'passed', 'failed', or 'pending'. Use *ngSwitch in CourseCardComponent to display a different badge for each status: green 'Passed', red 'Failed', grey 'Pending'.

28. Add a *ngIf with an else block: <ng-template #noCourses><p>N- courses available.</p></ng-template>. Show this template when the courses array is empty.

Hint:

- *ngIf with else uses Angular's ng-template ? the else block is never added to the DOM when the condition is true, unlike CSS display:none which hides but keeps the element.

- trackBy is essential for large lists ? without it, Angular re-renders every list item on any array change. With trackBy, only changed items are updated.

Expected Outcome: Loading message shows for 1.5 seconds then disappears. trackBy is implemented.

Switch renders the correct badge per grade status. Empty array shows the no-courses template.

Task 2: Attribute Directives ? ngClass and ngStyle

Goal: Apply dynamic CSS classes and inline styles using attribute directives.

Steps:

Angular HandsOn 3: Directives and Pipes

29. In CourseCardComponent, use [ngClass] to apply a CSS class conditionally: apply a 'card--enrolled' class when the course has been enrolled, and 'card--full' when credits ≥ 4 . Define both classes in course-card.component.css.
30. Use [ngStyle] to set the card's left border colour dynamically based on gradeStatus: green for 'passed', red for 'failed', grey for 'pending'.
31. Add an isExpanded property to CourseCardComponent. Use [ngClass]='{ "expanded": isExpanded }' to toggle a class that increases the card height. Toggle isExpanded on a 'Show Details' button click.
32. Refactor the ngClass object binding to use a getter in the component class: get cardClasses() { return { ... } } ? bind it as [ngClass]='cardClasses'. Explain in a comment why getters keep templates clean.

Hint:

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book

- [ngClass] accepts a string, array, or object. The object form { 'className': condition } is the most readable for conditional classes.

- Prefer [ngClass] over [ngStyle] for styling ? classes keep styles in CSS files where they belong. Use [ngStyle] only for truly dynamic values that cannot be expressed as a class.

Expected Outcome: Enrolled cards have a visible 'card--enrolled' style. Grade status is reflected by border colour. Show Details toggles the expanded class.

Task 3: Custom Directive and Custom Pipe

Goal: Build a reusable attribute directive and a custom pipe for the portal.

Steps:

33. Generate a custom directive: ng generate directive directives/highlight. The directive should add a yellow background to any element it is applied to on mouseenter, and remove it on mouseleave. Use @HostListener for both events.
34. Apply the directive to course cards in the template: <app-course-card appHighlight ...>. Verify the hover highlight works.
35. Generate a custom pipe: ng generate pipe pipes/credit-label. The pipe should transform a credits number into a human-readable string: 1 ? '1 Credit', 2+ ? '2 Credits'. Handle edge cases (null, 0) by returning 'N- Credits'.
36. Use the pipe in CourseCardComponent's template: {{ course.credits | creditLabel }}. Test with credits = 1, credits = 3, and credits = null.
37. Make the highlight directive configurable: add @Input() appHighlight = 'yellow' so the caller can pass a custom colour: <app-course-card appHighlight='lightblue'>.

Hint:

- @HostListener('mouseenter') binds to host element events without needing to manually add/remove event listeners ? Angular handles cleanup automatically.

Angular HandsOn 3: Directives and Pipes

- Pipes are pure by default (only re-run when the input reference changes). If your pipe needs to re-run on mutable data changes, set `pure: false` in the `@Pipe` decorator ? but use this sparingly as it impacts performance.

Expected Outcome: Hovering over a course card shows a yellow highlight. `{{ 3 | creditLabel }}` displays '3 Credits'. The configurable colour works via input binding.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book